C语言运算符03

C语言--流程控制语句

1.算术运算符（加减乘）

符号	说明
+	加法
-	减法
*	乘法
/	除法
%	取余

2.算数运算符（除法和取余）

除法（/）

```
printf("%.2lf\n",4.23/2);
```

取余（%）

```
printf("%.2lf\n", 4%3);
//通用细节：
//1.整数运算，结果一定是个整数（只取整书部分）
printf("%d\n", 10/3);
//2.小数计算，结果一定是个小数
printf("%.2lf\n", 10.0/2.0);
//3.整数和小数计算，结果一定是个小数
printf("%.2lf\n", 10./ 2.2);
//4.小数直接参与计算，结果可能不精确（）
printf("%lf\n", 3.33333333333333+6.66666666666666);
//除法的细节
//1.不能除以0
//printf("%lf\n", 10.0 / 0);
//取余的细节：
//1.运算的数据必须全部是整数
//printf("%.2lf\n", 6.1&3);
printf("%.2lf\n", 10%3);
//2.获取余数的正负与第一个数字保持一致
printf("%d\n", 10 % 3);
printf("%d\n", -10 % 3);
//3.不能除以0
```

3.练习-数值拆分

1.int取整

2.%取余

3.核心：对10进行操作

4.隐式转换（默认）

计算时,数据类型不一样不能直接运算,需要转成一样的,才能运算

把一个取值范围小的,转成取值范围大的

取值范围:

char < short < int < long < long long < float < double

什么时候转换?

不同的数据类型进行计算、赋值等操作,会触发隐式转换

转换规则1:

取值范围小的,和取值范围大的进行运算,小的会先提升为大的,再进行运算

转换规则2:

char short类型的数据在运算的时候,都会直接先提升为int,然后再进行运算

5.强制转化

如果把取值范围大的,赋值给取值范围小的,就需要进行强制转换

格式:目标数据类型 变量名=(目标数据类型)被强转的数据;

例：int b=10 ;

short i=(short)b;

PS：1.对应的数据精度不能超过

2.整体要整体（）起来

`sizeof` **不会**因为赋值操作而改变对表达式的类型判断，它只看表达式本身的类型。赋值运算符右侧的表达式在编译时独立计算类型，然后才进行赋值转换，但 `sizeof` 不会“穿透”赋值去查看左侧变量的类型。 `只会读，不会=`

6.字符相加

1.char short类型的数据在运算的时候,都会直接先提升为int,然后再进行运算

2.字母对应ASCALL码表，所以会升为int

```
printf("%d\n",'a');
printf("%d\n",'A');
printf("%d\n",'o');
```

直接测

- `'a'`（单引号）：

    - 在 C 语言中，它的**类型是** `int`（整数）。
    - 编译时，编译器直接把它替换成对应的 ASCII 码数值（即 97）。
    - 所以 `printf("%d", 'a')` 等价于 `printf("%d", 97)`，打印结果完全正确。
- `"a"`（双引号）：
    - 它的**类型是** `char*`（字符指针），即一个指向内存地址的指针。
    - 它表示在内存中存储了字符 `'a'` 和末尾结束符 `'\0'` 的这块连续空间的**首地址**（比如 `0x7ffd1234`）。
      7.自增自减运算符

符号	说明
++	变量中的值+1
--	变量中的值-1

PS:++和 -- 既可以放在变量的前边,也可以放在变量的后边
运用场景：
年龄，购物车
扩展：（只考）
++a先因后果，++a先果后因（加减针对原来的值）（++前都加，后++只变原来的）
//**windows前缀>计算>后缀**
//前缀统一进行自增/自减，再把结果拿出来用
//int i=10;
// ++i + ++i;
// 12+12=24
//后缀统一先用，全部变量用完了再自增/自减
//int i=10;
// i++ + i++;
// 10+10=20
// i=12
//mac,linux
//前缀后缀优先级一样
//从左到右，独立进行
8.赋值运算符

符号	说明
=	直接赋值
+=	加后赋值

符号	说明
-=	减后赋值
*=	乘后赋值
/=	除后赋值
%=	取余赋值

a /= b;（无int）

9.关系运算符（比较运算符）

符号	说明
==	判断a是否等于b
！=	判断a是否不等于b
>	判断a是否大于b
>=	判断a是否大于等于b
<	判断a是否小于b
<=	判断a是否小于等于b

PS：1成立，2不成立

10.逻辑运算符

作：连接多个表达式（∪）

符号	作用	说明
&&	与（而且）	两边都为真，结果才是真（有假就假）（同时满足）
||	或（或者）	两边都为假，结果才是假（有真为真）（二选一）
!	非（取反）	取反（！=）

用电路的思想：与是开关串联，或是开关并联，（1是闭合，0是断开）

PS:短路效果

```
int a = 1, b = 5;
a >10 && ++b;
printf("x=%d, y=%d\n", a, b);
```

如果&&左侧已经不成立就不执行右侧的代码了-

11.三元运算符（？：表达式）
格式:关系表达式?表达式1:表达式2;
（成立结果为1，不成立结果为0）
(目前只能用数字运算)
12.逗号运算符（分隔符）
规则:

1. 从左到右
2. 最后一个子表达式的结果,是整个表达式的结果
   3.优先级最低(永远是最后执行)
   ps:num1 >= 0 ? num1 : -num1;9绝对值的表达
   12.运算符的优先级
   1.小括号优先于所有

2.一元>二元>三元
（元为变量
3.&&>||>赋值>，（() 最高，最低）
4.先计算后判断
5.非0都成立
嵌套（从左边的**几个？起到右侧的几个：f**截至一次划分呢个，内部之后再划分）(?:间划分)